Design and Implementation of a Ride Sharing Platform Using Graph and Greedy Algorithm in C

Shaik Mynuddin
Department of Computing and Data Science
Sai University

Thrli Santhosh Syam Prasad
Department of Computing and Data Science
Sai University

***Abstract*—This paper presents the design and implementation of a Ride Sharing Platform developed using the C programming language. The system efficiently manages drivers and passengers and matches them based on minimum distance using graph representation and a greedy algorithm. Drivers and passengers are stored using structured arrays, while distances between them are represented using an adjacency matrix. The system dynamically assigns the nearest available driver to a passenger to optimize ride allocation. Additional functionalities such as adding, deleting, and displaying users are included. This implementation demonstrates the use of fundamental data structures and algorithms to solve real-world optimization problems.**

***Index Terms*—Ride Sharing, Graph, Greedy Algorithm, Adjacency Matrix, C Programming, Optimization**

I. Introduction

Ride sharing systems play a crucial role in modern transportation by connecting drivers and passengers efficiently through digital platforms. These systems aim to reduce travel time, fuel consumption, and traffic congestion by optimizing resource utilization. The key challenge lies in matching drivers with passengers in a way that minimizes distance while maintaining fairness and efficiency. This project presents a simplified ride sharing platform implemented in C using fundamental data structures and algorithms. The system leverages graph representation and a greedy algorithm to ensure efficient matching.

II. Problem Statement

The objective of this system is to design a ride sharing platform that assigns drivers to passengers based on minimum distance. The system must manage multiple users, maintain accurate distance data, and ensure that each passenger is assigned to only one driver. It should support dynamic operations such as adding and deleting users while maintaining system efficiency and reliability.

III. Data Structures Used

The system uses structures to represent drivers and passengers, storing attributes such as ID, name, and availability. Arrays are used to store multiple records efficiently. An adjacency matrix represents distances between drivers and passengers, where each element indicates the cost between a pair. A constant value is used to represent infinite distance when no connection exists.

IV. System Design

The system follows a modular design consisting of separate functions for adding drivers, adding passengers, creating edges, displaying data, and matching. Each entity is uniquely identified. The adjacency matrix forms a bipartite graph between drivers and passengers. A menu-driven interface allows user interaction and ensures ease of use.

V. Algorithm Used

A greedy algorithm is used for matching drivers and passengers. For each available driver, the system selects the nearest unassigned passenger by checking minimum distance in the adjacency matrix. Once assigned, the passenger is marked and the driver becomes unavailable. This approach ensures fast and efficient allocation but may not guarantee global optimality.

VI. Implementation Details

The system is implemented in C using modular functions. Key operations include adding users, inserting edges, displaying data, and performing matching. The adjacency matrix is initialized with infinite values. The program uses a menu-driven interface for interaction and supports deletion operations while maintaining consistency.

VII. Results and Discussion

The system successfully assigns drivers to passengers based on minimum distance. It ensures unique assignments and considers only available drivers. The greedy approach provides fast results suitable for real-time systems, though it may not always produce globally optimal solutions.

VIII. Advantages

The system is simple, efficient, and easy to implement. It demonstrates practical use of graphs and greedy algorithms. It supports dynamic updates and provides fast matching, making it suitable for small-scale applications and educational purposes.

IX. Limitations

The greedy algorithm does not guarantee optimal solutions in all cases. The system uses static arrays, limiting scalability. The adjacency matrix may become inefficient for large datasets, and the system lacks advanced features like real-time tracking and route optimization.

X. Conclusion

This project demonstrates the application of data structures and algorithms in solving ride sharing problems. The use of graph representation and greedy matching provides an efficient solution. Future improvements can include advanced algorithms like the Hungarian method and dynamic memory allocation for scalability.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.

[2] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *Data Structures and Algorithms*.

[3] B. Kernighan and D. Ritchie, *The C Programming Language*, Prentice Hall.

[4] S. Sahni, *Data Structures, Algorithms, and Applications in C++*.